Ulster University

JavaScript

Part One

ulster.ac.uk

Objectives

- To discover the reasons to learn JavaScript.
- To learn the history of JavaScript.
- To master the fundamental concepts.
- To become familiar with the Document Object Model (DOM).

Ulster University

Introduction

- Broad support among web browsers.
- Vast libraries of scripts available online.
- Applicable to other host environments.
- Allows use of reusable code libraries.
- Similar syntax to C, C++, and Java.
- Encourages creative problem solving.

Ulster University

Warning!

JavaScript is **NOT** Java!

Ulster University

JavaScript Characteristics

- An object-orientated programming language.
- JavaScript code is integrated directly into common HTML documents.
- Interpreted by the web browser when the document is loaded.
- Weakly typed.

Ulster University

Use Cases

- Two use cases – Client and Server side.
- Client side:
  - Program runs on client (browser).
  - A wrapped/progressive web app.
- Server side:
  - Program runs on a server.

Ulster University

Core Facets

- Three core facets of browser based JavaScript:
  - Core JavaScript.
  - Document Object Model (DOM).
  - Browser Object Model (BOM).

Document Object Model

- What is the DOM?
  - Internal road map of objects on a web page, allows programmers generic access – adding, deleting, and manipulating of all the styles, attributes, and elements in a document.
  - A platform and language neutral interface that will allow programs and scripts to dynamically access and update the content, structure, and style of documents.
  - Is like to be best suited for applications where the document must be accessed repeatedly or out of sequence order.

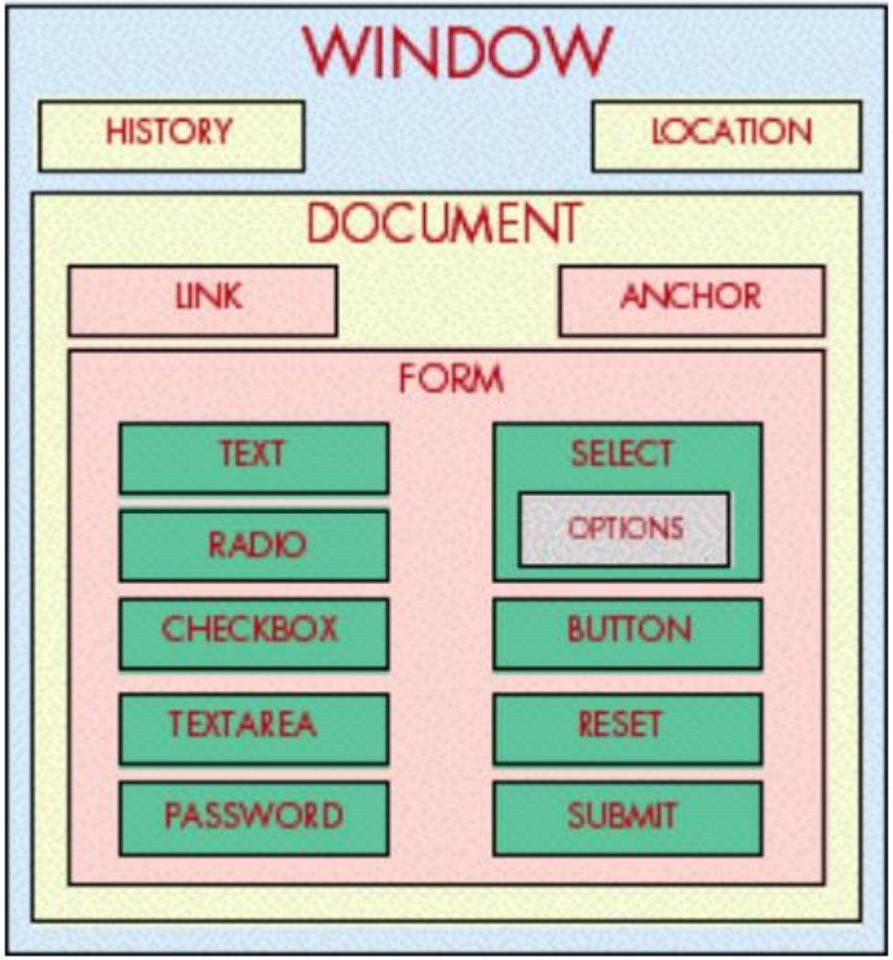
Document Object Model

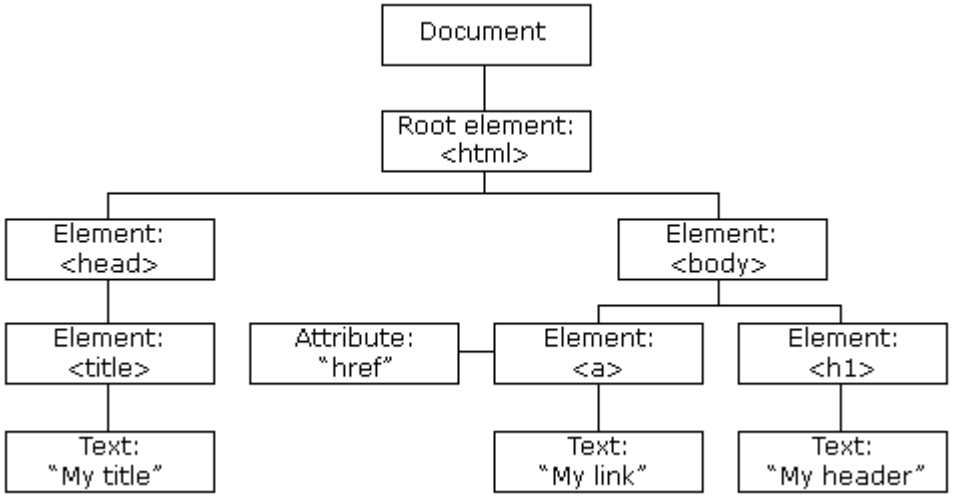
Objects, Properties, Methods, Functions, and Events

- All the elements that go into making a page are **objects**. (forms, images, tables, *etc.*)
- All objects have **properties** (colour, length, size, *etc.*) which **values** that define them.
- Actions that your script performs on or with objects are called **methods** (alert, write, *etc.*). These are **functions** built into objects.
- Dealing with user controlled **events**, like clicks and mouse over, is accomplished through commands called **event handlers** (onclick, onhover, *etc.*)

Ulster University

Document Object Model





Document Structure

HTML

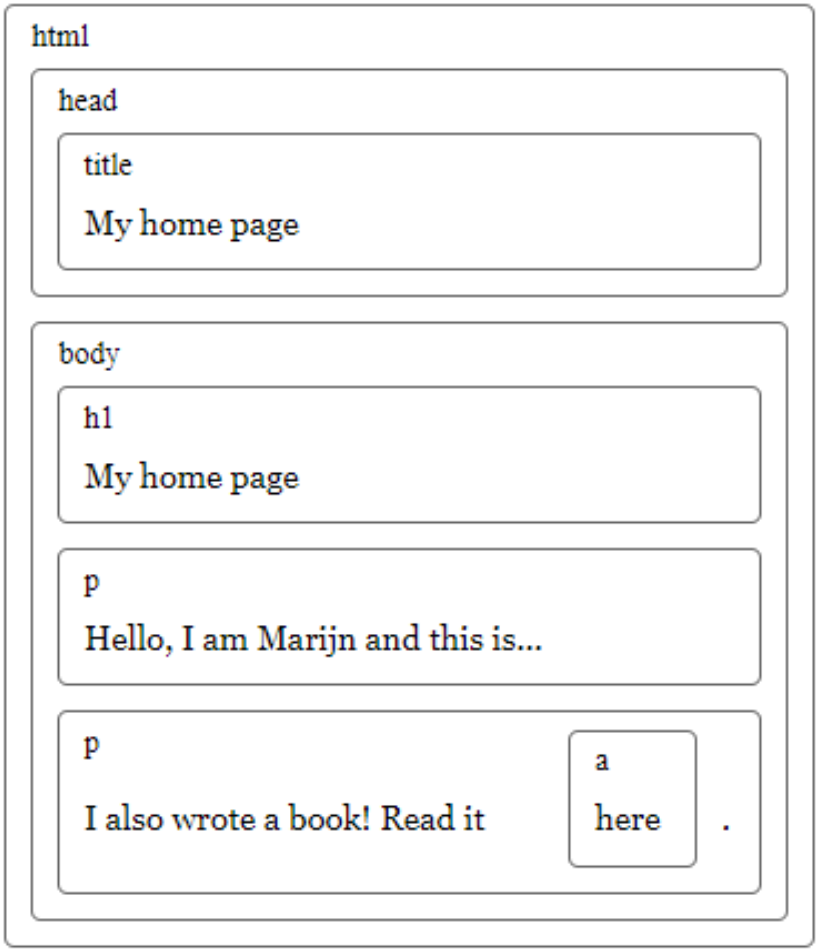
- You can imagine an HTML document as a set of nested boxes. Tags such as <body></body> enclose other tags, which then enclose additional tags.

```
<!doctype html>
<html>
  <head>
    <title>My home page</title>
  </head>
  <body>
    <h1>My home page</h1>
    <p>Hello, I am Marijn and this is my home page.</p>
    <p>I also wrote a book! Read it
      <a href="http://eloquentjavascript.net">here</a>.</p>
  </body>
</html>
```

Ulster University

Document Structure

- This representation is knows as the DOM.

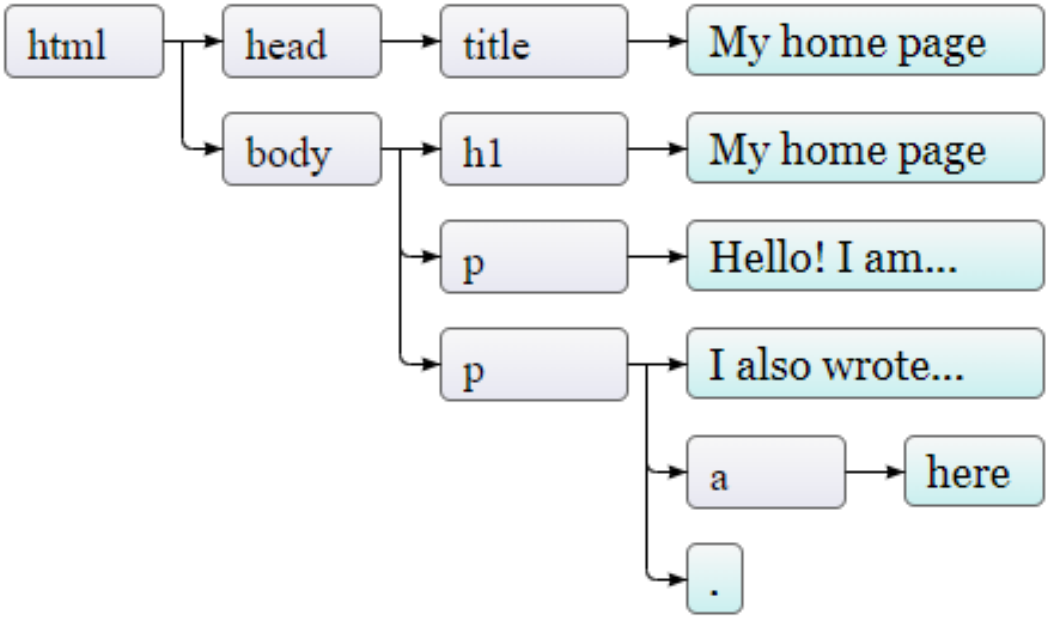



```
<!doctype html>
<html>
  <head>
    <title>My home page</title>
  </head>
  <body>
    <h1>My home page</h1>
    <p>Hello, I am Marijn and this is my home page.</p>
    <p>I also wrote a book! Read it
      <a href="http://eloquentjavascript.net">here</a>.</p>
  </body>
</html>
```

Document Structure

Root

- We call a data structure a *tree* when it has a branching structure, and has a single, well-defined *root*.
  - In the case of the DOM, document.documentElement serves as the root.



Ulster University

Browser Object Model

DOM

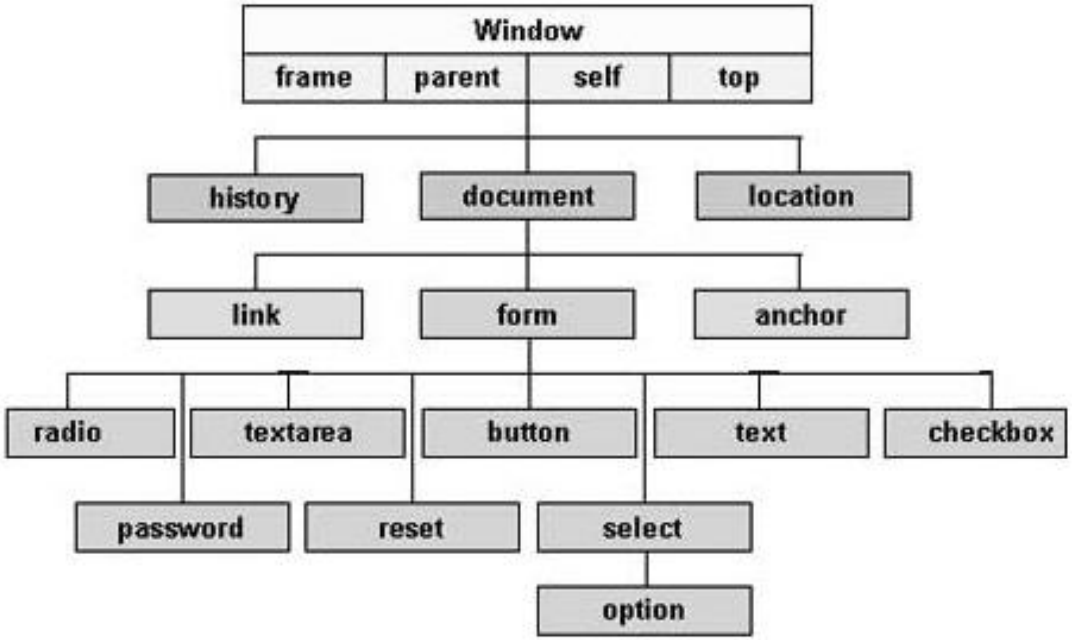
- Browser Object Model – The collection of objects that define the browser window and its content for the browser software.
- Allows JavaScript to interface and interact with the browser itself.
- Think of it as a level higher than the DOM.

Ulster University

Browser Object Model

Properties and Methods

- It's main purpose is to manipulate the properties and methods which are associated with the web browser.



Ulster University

Browser Object Model

BOM Hierarchy

- Window Object:
  - Represents the window which is open in the browser at the time – contains all other objects within.
- Document Object:
  - The Document Object Model.
- Navigator Object:
  - Used for detecting the browser information like the appName, appVersion, *etc.*
- History Object:
  - Contains all the information related to the browser history.

Browser Object Model

BOM Hierarchy

- Location Object:
  - Provides information regarding the current URL opened on the browser.
- Screen Object:
  - Provides information about the browser screen, such as height, width, *etc.*

DOM and BOM

- DOM gives access to each and every element in an electronic document (be the document HTML or XML).
  - The programmer just needs to call the element by its id or by it's position in the document to change any element dynamically across browsers.

- BOM makes objects of elements that are proper to the browser.
  - We are using BOM whenever we open a new window or whenever we create an alert or create a prompt or whenever we alter the status bar message, *etc.* Those objects belong to the browser.

Ulster University

Events and Event Handlers

- HTML/JavaScript events and event handlers can be something a browser or user does when a page is being loaded or interacted.
- These are defined within HTML and typically call JavaScript functions.
- Here are some examples of some types of events:
  - An HTML web page has finished loading.
  - An HTML input field was changed.
  - An HTML button was clicked.

Ulster University

Events and Event Handlers

- Events are defined within HTML in the following ways:

```
<element event="someJavaScript()">
                    or
<element event='someJavaScript()'>
```

Ulster University

Events and Event Handlers.

- Examples of these are:

```
<button  onclick="this.innerHTML  =  Date()">The
time is?</button>
```

or

```
<button
onclick="document.getElementById('displayDiv').i
nnerHTML = 'the button has been clicked'">CLICK
ME!</button>
```

Ulster University

Events and Event Handlers

onClick	Some action will be performed when any mouse button is clicked.
onLoad	Some action is performed when the document is loaded.
onMouseOver	Some action is performed when the mouse is moved over something.
onResize	Some action is performed when a page or element is resized.
onFocus	Some action is performed when a particular element gets focus.
onKeyDown	Some action is performed when a key is pressed over an element.
onKeyPress	Some action is performed when a key is pressed and released over an element.
onKeyUp	Some action is performed when a key goes up over an element.

Ulster University

Placing JavaScript

- Scripts can go in the HEAD or BODY:
  - Statements are placed in the HEAD within the <script></script> tag.

```
<script type="text/javascript"
  language="javascript">
Script body
</script>
```

  - Functions should be put in the HEAD, event handlers in the BODY can send values to functions in the HEAD.

Ulster University

Code Libraries

- Reuse your favourite scripts.
- Create your own functions and store as code libraries in plain text files
- Use the .js extension for code libraries.
- Reference your source libraries using the script tag.

```
<script type="text/javascript" language="javascript"
src="mylibrary.js">
```

Ulster University

Code Libraries.

```
<html><head><title>HTML Builder page</title>
<script type="text/javascript"
language="javascript" src="mylibrary.js">
</script>
</head>
<body bgColor="white">
<h1>HTML Builder</h1>
<p>Fill out the form below to create a basic
html page.</p>
<form name="pageform">
```

Ulster University

Fundamental Concepts

- Statements:
  - Terminated with a semicolon *e.g.* X = 5;
- Variables:
  - Types: numeric, string, Boolean, NaN (Not a Number).
- Variable naming conventions:
  - Start with an alpha numeric character.
  - Can contain numeric characters (not as first character).
  - Only allowed special characters is underscore (_).
  - Reserved words can **NOT** be used as variable names.

Ulster University

Variables

- Declare variables:
  - Using *var*
    - var myName;
    - var myAge;
  - Multiple variables using the same *var*:
    - var myName, myAge;
  - Declare variables with initial values:
    - Var myName = “john”;
    - Var myAge;
    - myAge = “24”;

Ulster University

Example Variable Names

Are these valid JavaScript variable names?

- time ✓
- 123test ✗
- x1 ✓
- squared? ✗
- _1st_counter ✓
- function ✗

Ulster University

Operators.

- Operators are comprised of assignment, comparison, arithmetic, string, and logical types.
- Assignment:

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

Operators

- Comparison:

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

Ulster University

Operators

- Logical:

Operator	Description
&&	logical and
||	logical or
!	logical not

Ulster University

Scope

- Variable scope:
  - Local – a local variable will be visible only within a function where it is defined. Function parameters are always local to that function.
  - Global – a global variable has global scope which means it is defined everywhere in your JavaScript code.

- Variable initialise:
  - Loosely typed – no need to declare types.

```
var myCampus = “Jordanstown”;
myCampus = 0;
myCampus = true;
```

Ulster University

Scope

Global Scope

- Globally declared variables can be accessed and modified anywhere in the code.
- Global variables are declared in the code outside of any function definitions, in the top level scope.

```
var a = 'Fred Flinstone'; // This is a global variable
function alpha() {
    console.log(a);
}
alpha(); // Outputs 'Fred Flinstone'
```

Ulster University

Scope

Global Scope

- When declaring variables in a web browser they are also properties of the global *window* object.

```
var b = 'Wilma Flintstone';
window.b = 'Betty Rubble';
console.log(b); // Outputs 'Betty Rubble'
```

- *b* can be accessed/modified as the property of the *window* object *window.b*.

- Be **very** careful when using global variables – they can lead to unreadable code that is also difficult to test.

Ulster University

Scope

Local Scope

- Variables declared within a function are scoped locally to that function.

```
var a = 'Daffy Duck'; // a global variable
function delta(b) {
    // b is a locally scoped variable to delta
    console.log(b);
}
function epsilon() {
    // c is declared as a locally scoped variable to epsilon
    var c = 'Bugs Bunny';
    console.log(c);
}
delta(a); // Outputs 'Daffy Duck'
epsilon(); // Outputs 'Bugs Bunny'
console.log(b); // Throws an error as b is undefined in the global scope
```

Ulster University

Scope

let

- *let* is seen by many to be a replacement for *var* but this isn't true.
  - *var* allows us to create a locally scoped variable.
  - *let* allows us to scope a variable to a block (a block being a set of curly braces)

```
function eta() {
    let a = 'Scooby Doo';
}
eta();
```

Ulster University

Scope

const

- *const* is the same in scope as *let* – in other words it is confined to the block of curly braces it is defined in.

- However, once a *const* has been defined it cannot be re-assigned.

```
if (true) {
  const a = 'Count Duckula';
  console.log(a); // Outputs 'Count Duckula'
}
console.log(a); // Outputs 'ReferenceError: a is not defined'
```

```
const b = 'Danger Mouse';
b = 'Greenback'; // Throws 'TypeError: Assignment to constant variable'
```

Ulster University

Scope

Global + Local

- When we redeclare a global variable in the local scope JS initiates a new local variable. The new local variable does not modify the global variable, but if we want to access the global value we need to use the window object.

```
var a = 'Johnny Bravo'; // Global scope
function iota() {
    var a = 'Momma'; // Local scope
    console.log(a); // Outputs 'Momma'
    console.log(window.a); // Outputs 'Johnny Bravo'
}
iota();
console.log(a); // Outputs 'Johnny Bravo'
```

Ulster University

Questions

Ulster University